

MSAGLJS: Browsing Large Graphs with Tile Pyramids and Sleeve Routing

Anonymous author(s)

Anonymous affiliation(s)

Abstract

We present a new way to visualize a large graph in the style of online geographic maps. The method builds a *tile pyramid* for *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on those maps.

The edges are routed by a method we call *sleeve routing*, which computes for each edge the shortest path in its homotopy class around the node obstacles, using the Constrained Delaunay Triangulation. We apply several heuristics to speed up the routing.

We implemented our approach in the WebGL renderer of MSAGLJS, an open-source TypeScript library for graph visualization in web browsers, with the entire pipeline running client-side, without a dedicated server. We target graphs with at most 40K nodes and 100K edges. The renderer's demo can be seen at <https://microsoft.github.io/msagljs/renderer-webgl/index.html>.

MSAGLJS is available on GitHub¹ and as NPM packages².

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,³ and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS⁴ is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme with semantic zoom.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the `deck.gl` framework [2]. It supports the layered Sugiyama scheme [44], Pivot MDS [14], and IPSep-CoLa [21], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The rendering pipeline builds a hierarchy of tiles—analogueous to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers`

¹ <https://github.com/microsoft/msagljs>

² `@msagl/core`, `@msagl/drawing`, `@msagl/parser`, `@msagl/renderer-svg`, `@msagl/renderer-webgl` — all on <https://www.npmjs.com/>.

³ In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2³² bytes).

⁴ <https://github.com/microsoft/msagljs>



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).

Editors: TBD; Article No. 0; pp. 0:1–0:13



Leibniz International Proceedings in Informatics

LIPIC Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

package, giving smooth pan and zoom. Live demos are available online: a WebGL renderer for large graphs⁵ and an SVG renderer for smaller, editable graphs⁶.

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *sleeve routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

2.0.0.1 Graph visualization tools.

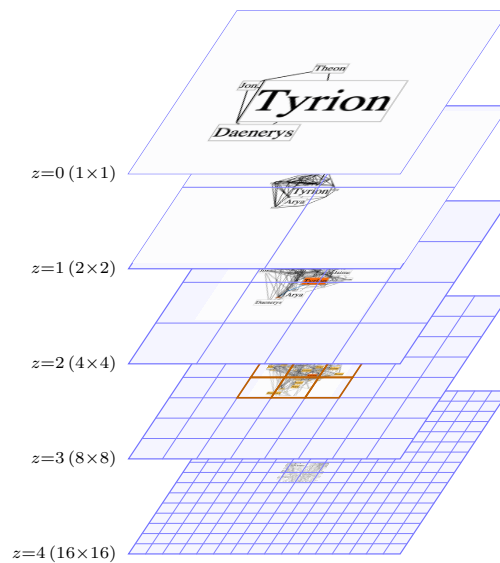
Graphviz [25, 5] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [7]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [39] that did not have the features described here. On the web, Sigma.js [8] and Cytoscape.js [23] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [6] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [1] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [38] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [41] handles huge graphs with tiles and edge bundling [34, 32] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [12], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [24], which distort a multilevel layout [27, 33] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [31] and force-directed bundling [32] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our sleeve algorithm walks the dual of a Constrained Delaunay Triangulation [43] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [36, 15, 26, 4]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [30] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [20] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [18] but requires a simple containing polygon. yWorks [11] offers “Organic edge routing” based on force-directed simulation. The approach of Dwyer and Nachmanson [22] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [45]. Our sleeve method eliminates the spanner entirely and routes

⁵ <https://microsoft.github.io/msagljs/renderer-webgl/index.html>

⁶ <https://microsoft.github.io/msagljs/renderer-svg/index.html>



121 **Figure 1** The entities rendered on each level as the user zooms in shown without scaling.

73 directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm
74 integrated with a browser-side tile pyramid that reroutes edges per level.

75 2.0.0.3 CDT-based pathfinding.

76 The closest precedent to our routing pipeline is the CDT-and-funnel pathfinder of Demyen
77 and Buro [17], originally designed for game-AI units of nonzero radius. They run admissible
78 A* search on the CDT dual (with a reduced-graph variant that collapses corridor chains into
79 single abstract edges) to recover provably shortest paths. Their search is, however, per-query:
80 the bounds depend on the start and goal, so it cannot share work across the edges with the
81 same source as our per-source Dijkstra does. For the graphs we target this would be roughly
82 an order of magnitude slower; we therefore trade their proven optimality for speed.

83 3 Tiling

84 The input to tiling is a laid-out graph together with routed edges: we prefer route the edges
85 with sleeve routing (Section 4), to achieve the visual stability on a level change. The output
86 is a sequence of $Z+1$ levels: level Z is the finest, and level 0 is the coarsest. Each level is a
87 collection of tiles indexed by integer (x, y) grid coordinates. The output is consumed by the
88 `TileLayer` of the `@deck.gl/geo-layers` package [10].

89 A tile is a rectangular viewport together with its *tile data*: nodes, edge labels, edge
90 arrowheads, and *edge clips*. An edge clip is a curve confined to the tile rectangle, paired
91 with the non-empty list of graph edges whose routes coincide with that curve inside the tile.
92 Bundles capture the stretches on which several routes travel tight together inside a common
93 sleeve; representing each such stretch as a single clip with an attached edge list, rather than
94 as one clip per edge, lowers each tile's element count and the number of draw calls at render
95 time, and avoids the extra subdivisions that an inflated count would otherwise trigger.

120 Pseudocode 1 describes the algorithm building the tile pyramid.

122 The single tile at level 0 has a rectangle equal to the graph bounding box. Each coarser

96 **Algorithm 1** BUILD_TILE_PYRAMID($G, \mathcal{C}, \text{minTileSize}, \mathcal{M}_{\max}$)

```

97 1:  $T_0 \leftarrow$  single tile holding all of  $G$  clipped to its bounding box
98 2:  $\text{levels} \leftarrow [T_0]$ ;  $z \leftarrow 0$ ;  $\text{used} \leftarrow |T_0|$   $\triangleright |T|$  = element count of tile  $T$ , summed over all
99   built levels
100 3: while some tile in  $\text{levels}[z]$  exceeds  $\mathcal{C}$  elements and its width/height  $\geq \text{minTileSize}$  do
101 4:    $\text{next} \leftarrow$  split every tile of  $\text{levels}[z]$  into four sub-tiles
102 5:   within each parent, distribute nodes/labels/arrowheads by bbox, and split each edge
103   clip at the tile midlines (Section 3.3)
104 6:   if  $200 \cdot (\text{used} + |\text{next}|) > \mathcal{M}_{\max}$  then
105 7:     discard  $\text{next}$ ; break  $\triangleright$  memory budget reached; check is incremental during the
106     split
107 8:   end if
108 9:    $\text{levels}[z+1] \leftarrow \text{next}$ ;  $\text{used} += |\text{next}|$ ;  $z \leftarrow z + 1$ 
109 10: end while
110 11:  $Z \leftarrow z$ ;  $V_Z \leftarrow V(G)$ ;  $s_v^{(Z)} \leftarrow 1$  for all  $v \in V_Z$ 
111 12: compute PageRank( $G$ )
112 13: for  $z = Z - 1$  down to 0 do
113 14:    $(V_z, \{s_v^{(z)}\}) \leftarrow \text{SELECT\_TOP\_K\_WITH\_ADAPTIVE\_SCALE}(V_{z+1}, 2^{Z-z})$   $\triangleright$  Section 3.1
114 15:   build a CDT from the inflated polylines of  $V_z$  at scales  $\{s_v^{(z)}\}$ 
115 16:   route every edge  $e \in E_z$  on this CDT, optionally using the level- $(z+1)$  route as a
116   sleeve hint  $\triangleright$  Section 3.2
117 17:   clip each edge of the level with the tiles
118 18: end for
119 19: return  $\text{levels}$ 

```

123 level keeps a prefix of the PageRank-ordered nodes together with all edges whose endpoints
124 both lie in that prefix; the finest level keeps the full node and edge sets.

125 3.0.0.1 How the index of the finest layer, Z , is chosen.

126 We treat the tiles differently when calculating Z and when preparing for rendering. There is
127 no element filtering in the former case: all graph entities are represented in each level. We
128 continue increasing Z , generating new levels, and split tiles, until a stopping condition is
129 met. Then, while keeping the finest level intact, we rebuild the tiles of levels $0, \dots, Z - 1$ for
130 rendering.

131 We start from $Z = 0$ and grow the pyramid one level at a time. We stop as soon as any
132 of three conditions is met:

- 133 1. every tile at the new level contains at most $\mathcal{C}$ elements, with default $\mathcal{C} = 500$;
- 134 2. the new tile width and height are both below ten times the average node width and
135 height;
- 136 3. while generating the tiles of the new finest level, the running total of stored tile elements
137 summed over all levels, multiplied by 200 bytes per element, exceeds the memory budget
138 $\mathcal{M}_{\max}$, with default 4 GB; the partial level is then discarded and Z is set to the previous
139 finest level.

140 The third condition is checked incrementally inside the level-build loop, so we abort as
141 soon as the memory barrier is reached rather than finishing a level we know we will throw
142 away. We additionally cap $Z \leq 8$ to bound the depth of the pyramid; the nine-level limit is
143 comfortably above what is reachable on any input we tested.

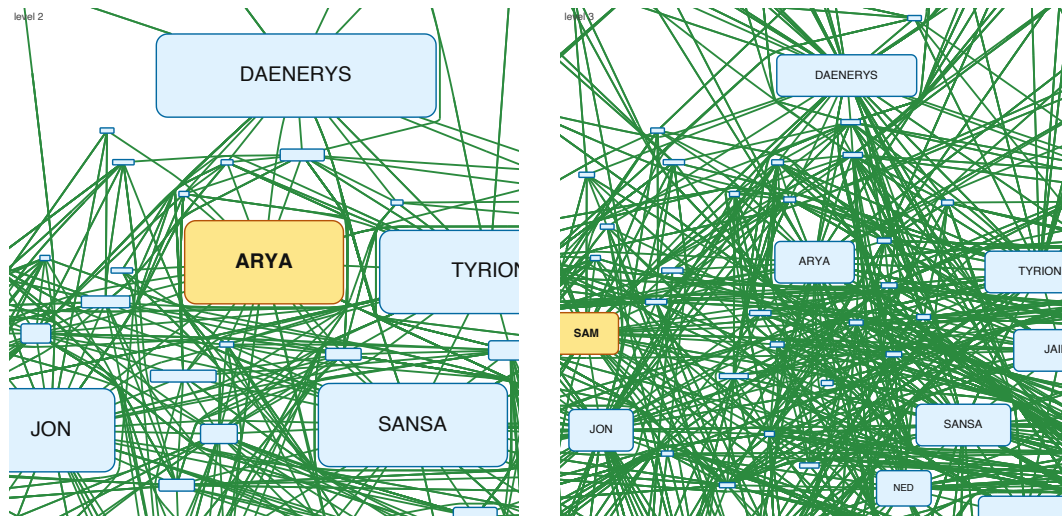


Figure 2 Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes.

3.0.0.2 Three phases.

Once Z and the tile rectangles are fixed, each coarser level $z = Z-1, Z-2, \dots, 0$ is built in three phases:

1. pick the nodes of level z with PageRank and possibly enlarge them, Section 3.1;
2. reroute the edges of G_z around the resulting obstacles, Section 3.2;
3. recompute the per-tile edge clips by splitting the new curves at tile boundaries, Section 3.3.

Figure 2 shows two adjacent pyramid levels built by this procedure; Algorithm 1 summarizes the whole pipeline.

3.1 PageRank-Guided Level Construction

Before filling the level's tiles for rendering we run PageRank [40] on G once, and sort all nodes of the graph, V , by descending rank. Write $k = Z - z$ for the level's depth below the finest level. The candidates for level of depth k are the nodes of the *prefix* of the sorted array of length $\lceil |V|/2^k \rceil$.

Node positions are inherited from the original layout and never move: across levels we change only each node's display size.

We add nodes to a level of depth k by walking the level's prefix in rank-descending order. The top-ranked node is scaled by 2^k . We keep the node scales non-increasing, but as large as possible, while making sure that each next node does not overlap the previously selected nodes. If even at its original size the candidate's bounding box would overlap an already-accepted node's scaled box, the candidate is dropped.

Dropping a relatively high-ranked candidate at a coarse level is less disruptive than it sounds. Consider Arya's node in the *Game of Thrones* graph: as Figure 1 shows, it is dropped from the top layer because a higher-ranked node, Tyrion, has covered the available region. The user still sees that part of the graph as occupied, and Arya's individual node appears at a finer level where its box fits.

172 3.1.0.1 Spatial-hash overlap test.

173 A naive overlap test against every previously accepted node would dominate the cost of the
 174 filter. We instead maintain the accepted boxes in a uniform spatial hash whose cell size is
 175 chosen so that any accepted box, and any candidate's box at the level's maximum scale,
 176 both fit inside one cell on each axis. Because of this choice, two such boxes can overlap only
 177 if their center cells differ by at most one step on each axis: testing a candidate therefore
 178 reduces to scanning the accepted boxes registered in the candidate's own cell and in the eight
 179 cells immediately around it.

180 The hash brings little benefit at the coarsest levels, where the prefix is short and even
 181 a naive scan over the few accepted boxes is cheap; it pays off at finer levels, where the
 182 prefix grows geometrically and a per-candidate constant-time test replaces a linear scan
 183 over hundreds of already-accepted boxes. On the layouts we tested the expected work per
 184 candidate is constant, so the overall cost is dominated by the initial sort by PageRank. In
 185 our experiments this phase was never a bottleneck: the per-level rerouting of Section 3.2
 186 dominates the build time at every level we measured.

187 3.2 Stable Per-Level Rerouting with a Routing Hint

188 At each level we change the graph and the node sizes, so the edges must be rerouted on each
 189 level as well. We rerun the sleeve router, Section 4, on the set of the level edges.

190 3.2.0.1 Per-level CDT.

191 For level $z - 1$ we build a dedicated CDT from the inflated polylines of V_{z-1} only, using
 192 each node's level- $z-1$ scale. The global landmark v_0 , for example, enters the triangulation as
 193 an obstacle that is 2^{Z-z+1} times its finest-level size in each dimension, so routes naturally bend
 194 around it.

195 3.2.0.2 Routing hint.

196 For every edge $e \in E_{z-1}$ we already have a route c_e computed at the finer level z . Sampling c_e
 197 and listing the level- $z-1$ CDT triangles it visits, expanded by one neighbor hop, yields a
 198 triangle *sleeve* S_e that we can use as a per-edge hint for an A* search restricted to S_e . This
 199 keeps the coarse route close to the fine one, so an edge deforms smoothly as the user zooms
 200 out instead of flipping to the other side of a landmark. The price is one independent A* per
 201 edge.

202 By default we instead reuse the per-source Dijkstra trees of Section 4.2, which are faster
 203 and behave well in practice. The reason this option is not always available is that, in a
 204 subgraph, the set of transparent obstacles is per-edge rather than per-source: the boundaries
 205 of nodes on the ancestor chains of the edge's endpoints may be crossed, and for edges sharing
 206 a source these chains differ with the target. A single Dijkstra tree from the source therefore
 207 cannot serve all of its outgoing edges in this setting, so the hint+A* variant is used in this
 208 case. The graphs listed in the paper do not have subgraphs.

209 3.3 Splitting Edges between Tiles

210 Once each level G_z has its final geometry, we cut that level into the tiles defined at the start
 211 of Section 3; we write $(rect, tiledata)$ for a tile, with *rect* its rectangle. Two edge clips inside
 212 the same tile are merged whenever their curves coincide on the tile's interior, with the second

Table 1 Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$, $Z_{\max} = 8$). The capacity $\mathcal{C}$ only guides the choice of Z ; we have no tight analytical bound on the actual rendered per-tile element count, which is therefore reported empirically. “Max tile’s level” is the level on which the maximum tile was found. In the worst cases, **ca-HepPh** and **ca-CondMat**, the densest tile holds $6\text{--}8\times$ the nominal capacity $\mathcal{C}$.

Graph	$ V $	$ E $	Levels built	Max per tile	Max tile’s level
gameofthrones	407	2 639	5	324	4
composers	3 405	13 832	8	749	3
ca-GrQc	5 242	28 968	8	359	4
ca-HepTh	9 877	51 946	8	1 836	3
facebook_combined	4 039	88 234	9	928	5
ca-HepPh	12 008	236 978	9	3 643	3
ca-CondMat	23 133	186 878	8	2 949	3
deezer_europe	28 281	92 752	8	2 402	3
delaunay_n15	32 768	98 274	8	283	7

clip’s edge list appended to the first’s; the implementation detects coincidence by snapping each clip’s tile-entry and tile-exit points to the layout tolerance and matching the resulting endpoint pairs. Because the sleeve router is deterministic, parallel edges sharing endpoints route along the same curve, so the merger is exact in the dense-supernode case it is most needed for.

The single tile at level 0 is obtained by clipping every element to the graph’s bounding box. To build level $z + 1$ from level z , each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines into sub-clips confined to individual sub-tiles, each sub-clip inheriting the parent’s edge list. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile’s rectangle, crossing the boundary only at its endpoints; (b) for every edge e , the union of all clips whose edge list contains e inside a tile equals $c_e \cap \text{rect}$. Part (a) makes subdivision cheap and local: thanks to (a) we never need to clip against the four outer sides of a tile, only against its two midlines. For the default polyline edges produced by the sleeve router this is trivial—each segment meets each midline at most once—so a clip is split by walking its segments and cutting at the midline crossings. The optional Bezier-corner pass of Section 4.3 replaces these line–line intersections with curve–line ones plus a parameter sort, but obeys the same invariant. Part (b) is the rendering-correctness condition: at any zoom only the tiles intersecting the viewport are fetched and their clips drawn, and (b) guarantees that the union of what is drawn equals $c_e \cap \text{viewport}$ for every visible edge e . The same invariant makes the rerouting of Section 3.2 compatible with tiling: whenever an edge curve is replaced by a coarser-level route, (b) is reestablished by recomputing the affected clips.

Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively. These are the same conditions that determine Z (Algorithm 1).

Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

256 4 Sleeve Routing on the CDT

257 We describe an edge routing method that routes edges directly on the dual graph of a
 258 Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a
 259 *sleeve*—a sequence of adjacent CDT triangles from source to target—and applies the funnel
 260 algorithm to produce the shortest path within it.

261 4.1 CDT Construction and the Dual Graph

262 Given the graph layout, each node is covered by a padded obstacle polygon. We compute a
 263 Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [16, 19]. The
 264 *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing
 265 a side. A triangle is *free-space* if it is not contained in any obstacle interior.

266 4.2 Routing inside a Sleeve

267 Intuitively, the sleeve is a chain of triangles meeting along shared edges, the *diagonals*, that
 268 the path must cross in order. Routing an edge from s to t splits into two subproblems:
 269 a *combinatorial* one—which sequence of diagonals to cross—and a *geometric* one—which
 270 exact polyline through that sequence is shortest. The geometric step is the classical funnel
 271 algorithm [15, 29], which pulls the path taut inside the polygon formed by the sleeve triangles.
 272 The sleeve router itself is therefore agnostic to how the sleeve is produced; any search on the
 273 dual graph that returns such a sequence of triangles will do.

274 To obtain the sleeve we run a shortest-path search on D where the step from one triangle
 275 to a neighbor costs the Euclidean distance between their centroids; triangles inside obstacles
 276 other than the source and target obstacles are excluded. The natural per-edge instantiation
 277 is A^* [28] with the straight-line distance to t as heuristic. When a node has many incident
 278 edges, however, running independent searches is wasteful. We instead group edges by source s
 279 and run a single Dijkstra computation per source, expanding until all targets are reached
 280 and recovering each sleeve by following parent pointers. This reduces the number of searches
 281 from m to at most n , the number of nodes; on the Game of Thrones social network (407
 282 nodes, 2 639 edges) it cuts routing time by 30%.

283 Since the query edges are undirected on D , each can be served from either endpoint,
 284 so the number of distinct roots is in turn reducible by reorienting query edges to minimize
 285 that count—an instance of minimum vertex cover on the demand graph, well approximated
 286 in linear time by the greedy maximum-degree rule and used as a source-side speed-up in
 287 many-to-many shortest-path frameworks [35]. Applying this heuristic reduces the number of
 288 Dijkstra trees from 331 (one per distinct source) to 199 on Game of Thrones and from 2 645
 289 to 1 285 on the composers graph (3 405 nodes, 13 832 edges), shortening the routing pass by
 290 $\sim 17\%$ and $\sim 35\%$ respectively.

291 Before launching the sleeve search for an edge (s, t) we attempt an even cheaper shortcut:
 292 starting at the triangle containing s , we walk the triangulation along the segment $\overline{st}$ triangle
 293 by triangle and check whether any traversed triangle is interior to a third obstacle. If none
 294 is, the straight segment is the geodesic and we use it directly, skipping both the dual-graph
 295 search and the funnel.

296 4.3 Vertex Collapse at Source and Target

297 We improve the sleeve’s geometry in the situation depicted in Figure 3. The remedy is to
 298 *collapse* corner-hugging chain vertices to the corresponding endpoint. Walking the right

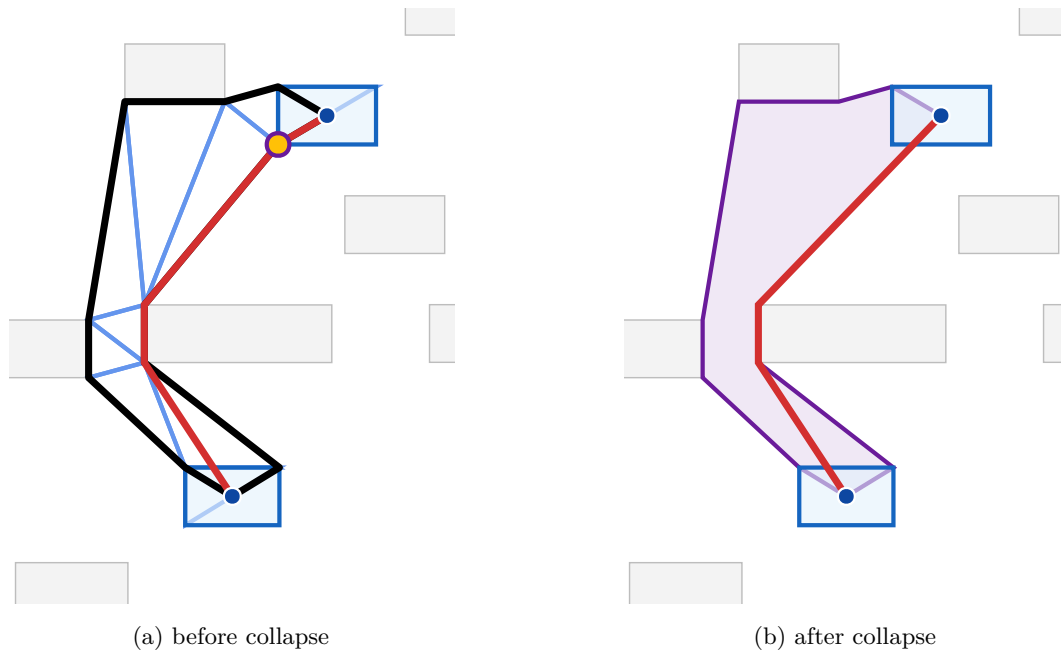


Figure 3 Effect of vertex collapse on the LORAS-RENLY edge of the Game of Thrones graph. Endpoints are shown as blue dots; their padded obstacles are highlighted in light blue. (a) The CDT triangles forming the sleeve are drawn in cornflower blue and the funnel path in red; the thick black polyline is the perimeter of the non-collapsed sleeve. The yellow dot marks the source-obstacle vertex that gets collapsed. (b) The sleeve after collapse, drawn as a single purple polygon with the source/target obstacles merged into the corresponding endpoints, and the funnel path (red) through it. Collapsing widens the funnel opening at the endpoint and removes the spurious detour around the obstacle corner.

chain of the sleeve outward from s , we find the first vertex of the source obstacle at which the chain turns right toward s , and replace every source-owned vertex on that chain up to and including this one by s . The left chain is processed mirror-symmetrically (with left turns instead of right), and the target end likewise. Diagonals whose two endpoints coincide after collapse are dropped, and the left/right orientation of each surviving diagonal is preserved from the uncollapsed geometry. The collapsed vertex in Figure 3(a) is marked in yellow.

After collapse, the funnel sees a wide opening at each endpoint, as in Figure 3(b), and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves. Collapse might cause the resulting path to overlap nodes other than the source and the target, but we have found the effect to be negligible.

The implementation also offers an optional pass that fits a Bezier segment into each polyline corner to render edges as smooth curves. It is disabled by default for performance: the polyline rendering is fast and visually acceptable, so we trade the smoother curves for routing speed.

5 WebGL Rendering

The WebGL renderer uses `deck.gl` [2], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView`, with no perspective distortion, and a `TileLayer` that dynamically loads tile data based on the current viewport. Because

deck.gl’s `TileLayer` expects a power-of-two pyramid of 512-pixel tiles, the root tile size is rounded up to the nearest power of two and a *start zoom* level is computed so that the root tile maps to deck.gl’s 512-pixel tile size. The `TileLayer`’s `getTileData` callback then translates deck.gl tile coordinates (x, y, z) to MSAGLJS tile map coordinates, fetching the precomputed tile data.

In deck.gl’s `OrthographicView` the screen scale at view zoom v is $s = 2^v$ pixels per world unit, and the start zoom is $v_0 = \log_2(512/W)$, so that at $v = v_0$ the root tile of width W projects to 512 pixels. The `TileLayer` then requests pyramid level $z = \lfloor v - v_0 \rfloor$, at which each tile covers $W/2^z$ world units and renders at about 512 pixels. Note v_0 lives in the same view-zoom space as v and is not a pyramid level: when the bounding box is bigger than one 512-pixel tile, v_0 is negative. For example, with a bounding box of 1000×800 world units rounded up to $W = 1024$ we get $v_0 = -1$; at $v = 2$ the scale is $s = 4$ px/wu, the chosen level is $z = 3$, and each level-3 tile spans 128 wu. A 1280×800 -pixel viewport then covers 320×200 wu and intersects $\lceil 320/128 \rceil \times \lceil 200/128 \rceil = 3 \times 2 = 6$ level-3 tiles, the only ones fetched (Figure 1). Between integer view zooms the level is held fixed and the tiles are GPU-scaled, so continuous panning and zooming never block on routing or rerouting.

5.0.0.1 Rendering layers.

Each tile is rendered as a composite deck.gl layer containing:

- A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).
- A **TextLayer** for node labels.
- A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at zoom level z , the resolution is 2^{z-2} segments per curve span.
- An **IconLayer** for edge arrowheads.
- A **TextLayer** for edge labels.

5.0.0.2 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

6 Experimental Results

We evaluate the sleeve router and the tile-pyramid build on the same benchmark graphs as Table 1: the Game of Thrones social network [13], the GD 2011 contest `composers` graph [9], four SNAP collaboration networks (`ca-GrQc`, `ca-HepTh`, `ca-HepPh`, `ca-CondMat`) and the SNAP `facebook_combined` ego network [37, 3], the `deezer_europe` social network [42], and the SuiteSparse Delaunay mesh `delaunay_n15`. All experiments run in Node.js (single process, headless; not a browser) on an Apple M4 Max laptop with 48 GB of unified memory. Layout uses IPsepCola, and the tile pyramid is built with the same parameters as Table 1, capacity $C = 500$ and $Z_{\max} = 8$.

Table 2 reports per-graph timings of three pipeline stages: *Routing*, the sum of the two phases logged by the sleeve router on the full graph—constrained Delaunay triangulation of the obstacles and the Dijkstra-tree search on the CDT dual (Section 4.2); *Tiling*, the build of the tile pyramid (Section 3), which includes the per-level rerouting of Section 3.2; and *Total*, the end-to-end loading time including parsing.

Graph	$ V $	$ E $	Routing	Tiling	Total
gameofthrones	407	2 639	0.17	0.17	1.87
composers	3 405	13 832	4.46	2.43	7.67
ca-GrQc	5 242	28 968	4.96	2.72	9.16
ca-HepTh	9 877	51 946	27.60	9.62	40.51
ca-HepPh	12 008	236 978	83.40	35.61	128.10
facebook_combined	4 039	88 234	13.82	6.55	22.33
ca-CondMat	23 133	186 878	202.50	55.29	273.60
deezer_europe	28 281	92 752	388.60	68.90	479.10
delaunay_n15	32 768	98 274	31.23	12.94	68.40

Table 2 End-to-end loading benchmarks on an Apple M4 Max laptop, all times in seconds. *Routing* sums the CDT-construction and Dijkstra-tree sleeve-search phases of Section 4.2 on the full graph; *Tiling* is the build of the tile pyramid (capacity $C = 500$, $Z_{\max} = 8$) of Section 3, which includes the per-level rerouting; *Total* is the total loading time including parsing of the graph file, graph layout, and all other stages.

The CDT phase contributes negligibly to the routing column—from 8 ms on Game of Thrones to about half a second on the largest graphs—so virtually all of *Routing* is the Dijkstra-tree sleeve search, with the vertex-cover heuristic of Section 4.2 reducing the number of trees we run. Even on graphs with hundreds of thousands of edges (**ca-CondMat**, **ca-HepPh**, **deezer_europe**) the full pipeline finishes in a few minutes; this is a one-time build cost. The resulting tile pyramid already contains the per-level routes of Section 3.2, so panning and zooming only fetch and render the precomputed tiles, with no rerouting at view time.

7 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The sleeve routing method routes edges directly on the CDT dual graph using batched Dijkstra trees and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at <https://github.com/microsoft/msagljs> under the MIT license.

References

- 1 Cosmograph. <https://cosmograph.app>.
- 2 deck.gl: Large-scale WebGL-powered data visualization. <https://deck.gl/>.
- 3 facebookcombined. https://snap.stanford.edu/data/facebook_combined.txt.gz.
- 4 Funnel algorithm. <https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf>.
- 5 Graphviz. <http://www.graphviz.org/>.
- 6 Regraph. <https://cambridge-intelligence.com/regraph/>.
- 7 sfdp. <https://graphviz.org/docs/layouts/sfdp/>.
- 8 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. <https://www.sigmajs.org/>.
- 9 Skewed. <http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml>.
- 10 TileLayer (@deck.gl/geo-layers). <https://deck.gl/docs/api-reference/geo-layers/tile-layer>. Accessed: 2026.
- 11 yworks. <https://yworks.com/products/yed>.

- 413 12 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale
414 graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*,
415 volume 12, pages 669–676. IEEE, 2006.
- 416 13 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concor-
417 dances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages
418 201–225. Routledge, 2018.
- 419 14 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional
420 scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe,*
421 *Germany, September 18-20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.
- 422 15 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium*
423 *on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.
- 424 16 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i*
425 *Estestvennyka Nauk*, 7(1):793–800, 1934.
- 426 17 Douglas Demyen and Michael Buro. Efficient triangulation-based pathfinding. In *Proceedings*
427 *of the 21st National Conference on Artificial Intelligence (AAAI)*, pages 942–947, 2006.
- 428 18 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implement-
429 ing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD’97*
430 *Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.
- 431 19 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation.
432 *International Journal of Geographical Information Science*, 22(4):449–462, 2008.
- 433 20 Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier
434 for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium*
435 *on Theory of Computing (STOC)*, 2025. <https://arxiv.org/abs/2504.17033>.
- 436 21 Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for
437 separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer*
438 *Graphics*, 12(5):821–828, 2006. doi:10.1109/TVCG.2006.156.
- 439 22 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing:*
440 *17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised*
441 *Papers 17*, pages 147–158. Springer, 2010.
- 442 23 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader.
443 Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–
444 311, 2016.
- 445 24 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing
446 large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468,
447 2005.
- 448 25 Emden R. Gansner and Stephen C. North. An open graph visualization system and its
449 applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233,
450 2000.
- 451 26 Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan.
452 Linear-time algorithms for visibility and shortest path problems inside triangulated simple
453 polygons. *Algorithmica*, 2(1–4):209–233, 1987.
- 454 27 David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In
455 *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.
- 456 28 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic
457 determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*,
458 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.
- 459 29 John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy
460 class. *Computational geometry*, 4(2):63–97, 1994.
- 461 30 John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in
462 the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.
- 463 31 Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical
464 data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.

- 465 32 Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization.
466 In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.
- 467 33 Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–
468 71, 2005.
- 469 34 Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density
470 estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library,
471 2012.
- 472 35 Sebastian Knopp, Peter Sanders, Dominik Schultes, Falk Schieferdecker, and Dorothea Wagner.
473 Computing many-to-many shortest paths using highway hierarchies. In *Proceedings of the 9th*
474 *Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 36–45. SIAM, 2007.
- 475 36 D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear
476 barriers. *Networks*, 14(3):393–410, 1984.
- 477 37 Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and
478 shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es,
479 2007.
- 480 38 Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd,
481 and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing*
482 *and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA,*
483 *September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.
- 484 39 Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In
485 *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.
- 486 40 Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation
487 ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.
- 488 41 Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data
489 infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.
- 490 42 Benedek Rozemberczki and Rik Sarkar. Characteristic Functions on Graphs: Birds of a
491 Feather, from Statistical Descriptors to Parametric Models. In *Proceedings of the 29th ACM*
492 *International Conference on Information and Knowledge Management (CIKM '20)*, page
493 1325–1334. ACM, 2020.
- 494 43 Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation.
495 *Computational Geometry*, 22(1–3):21–74, 2002.
- 496 44 Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding
497 of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*,
498 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.
- 499 45 Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture*
500 *Notes in Computer Science*, 5849:219–231, 2010.